



DISCRETE STRUCTURE

Bài Tập Lớn

BELLMAN-FORD ALGORITHM

Thảo luận BTL môn KTLT, DSA, NMLT, PPL
<https://www.facebook.com/groups/211867931379013>

Tp. Hồ Chí Minh, Tháng 3/2024



Mục lục

1	BELLMAN-FORD ALGORITHM (version 19/5)	3
2	Task 3 : Travel Saleman Problem	4
2.1	Mô tả bài toán:	4
2.2	Dynamic:	4
2.2.1	Mô tả giải thuật:	4
2.3	Ant colony:	5



1 BELLMAN-FORD ALGORITHM (version 19/5)

Mô tả các param

- **G** là Ma trận trọng số (weighted matrix) : Mỗi đơn vị trong ma trận biểu diễn khoảng cách tương ứng. Ví dụ $G[i][j]$ là khoảng cách từ thành phố i tới thành phố j . Trong trường hợp giá trị 0 nghĩa là không có đường đi trực tiếp từ thành phố i tới thành phố j .
- **numberOfVertices** là số lượng đỉnh trong đồ thị
- **startVertex** đỉnh bắt đầu
- **goalVertex** đỉnh kết thúc
- **Label** Một array chứa các giá trị khoảng cách ngắn nhất từ đỉnh bắt đầu đến mỗi đỉnh khác trong đồ thị. Giá trị này được cập nhật trong quá trình thực hiện thuật toán Bellman-Ford.
- **Prev** Một array chứa các đỉnh trước đó trên đường đi ngắn nhất từ đỉnh bắt đầu đến mỗi đỉnh khác trong đồ thị. Nó giúp xác định đường đi ngắn nhất từ đỉnh bắt đầu đến một đỉnh cụ thể.

```
void BF(int** G, int numberOfVertices, char startVertex,
        int* Label, int* Prev);
```

Mô tả BF

1. Cập nhật **Label[start]** = 0
2. Xét xem các **Label** có đang mang giá trị -1 hay không, nếu có thì gán giá trị cho **Label** bằng với các khoảng cách từ **start** đến các thành phố cùng với **Prev** là thành phố khởi đầu. Trường hợp khoảng cách tới thành phố = -1 thì **Prev** của thành phố đó vẫn là -1. Sau đó kết thúc hàm.
3. Khi **Label** đã không còn mang các giá trị -1 thì thực hiện tìm đường đi tốt hơn bắt đầu từ việc chép mảng **Label** để thực hiện các tính toán không bị mất đi số liệu cũ.
4. Duyệt tất cả các đỉnh **U** và so sánh xem **Label'[V] + G[V][U]** có nhỏ hơn **Label'[U]** hay không (V lần lượt là các thành phố trong bài toán) sau đó cập nhật giá trị **Label[U]** và **Prev** trong trường hợp nhỏ hơn. Nếu có 2 đỉnh cùng ra giá trị nhỏ hơn thì lưu ý lấy thành phố trung gian có giá trị Alphabet nhỏ hơn.

```
string BF_Path(int** G, int numberOfVertices,
               char startVertex, char goalVertex);
```

Mô tả BF Path

1. Khởi tạo **Label** và **Prev** cập nhật giá trị là -1.
2. Loop gọi hàm **BF** phù hợp để có thể lấy được mảng **Prev** đã tối ưu nhất.
3. Kiểm tra đường đi có tồn tại hay không nếu không thì print ra **return "No path exists";** và kết thúc hàm.
4. Có tồn tại thì dựa vào mảng **Prev** trả về chuỗi từ đỉnh đầu đến cuối không có dấu cách ở cuối các phần tử cách nhau bởi dấu cách.

Kiến thức cần có

- thư viện array <https://cplusplus.com/reference/array/array/>
- Lý thuyết Ma trận trọng số <https://users.monash.edu/~lloyd/tildeAlgDS/Graph/>
- BELLMAN-FORD ALGORITHM <https://www.youtube.com/watch?v=obWXjtg0L64>



2 Task 3 : Travel Saleman Problem

2.1 Mô tả bài toán:

Bài toán là tìm tổ hợp của các thành phố sao cho đường đi qua các thành phố 1 lần và quay lại thành phố ban đầu có khoảng cách nhỏ nhất.

2.2 Dynamic:

2.2.1 Mô tả giải thuật:

1. Phương pháp quy hoạch động trong bài này sẽ là Bellman-Held-Karp.
2. Ví dụ: Ta có các thành phố A B C D, và thành phố khởi đầu là A. Ta sẽ có các tổ hợp ABCDA, ABDCA, ACDBA, ACBDA, ADCBA, ADBCA.
3. Nếu tìm hết khoảng cách của các tổ hợp trên thì sẽ gọi là phương pháp vét cạn (thầy bảo có cách chống cách làm này), vì vậy sẽ tối ưu bài toán bằng cách chia nhỏ và tìm tối ưu.
4. Hướng làm:
 - (a) Gọi tập S là tập chứa các thành phố à A là thành phố khởi đầu
 - (b) Đường đi ngắn nhất sẽ được tính theo tổ hợp tối ưu hơn
 - (c) Ví dụ: 4 thành phố ABCD sẽ là A + tối ưu của BCD + A, trong đó BCD + A là khoảng cách nhỏ nhất của B + tối ưu của CD + A, C + tối ưu của BD + A, D + tối ưu của BC + A.
 - (d) Vì vậy ban đầu ta sẽ khởi tạo ma trận X $2^n * n$, với n là số thành phố, 2^n là số lượng tổ hợp, n là để lưu thứ tự đi của tổ hợp. Cùng với ma trận Y $2^n * n$ để lưu khoảng cách tối ưu của tổ hợp theo các điểm nối.
 - (e) Tính khoảng cách từ n đỉnh đi qua tổ hợp rỗng tới điểm khởi đầu.
 - (f) Ta thêm 1 thành phố vào tổ hợp trước đó để tính các khoảng cách nhỏ nhất dựa theo điểm nối và cập nhật giá trị.
 - (g) Lặp lại bước trên đến khi đã đi qua đủ tất cả các thành phố.
 - (h) Nếu không có đường đi thỏa mãn đi qua hết các thành phố thì **return "No path exists"**



Minh họa hướng thực hiện:

STT	X				Y								
	0	1	2	3	0	1	2	3		A	B	C	D
0100	0	1	0	0	0	0	0	0	A	0	1	3	1
0010	0	0	3	0	0	0	0	0	B	1	0	2	2
0001	0	0	0	1	0	0	0	0	C	3	2	0	3
0110	0	5	3	0	0	2	1	0	D	1	2	3	0
0101	0	3	0	3	0	3	0	1					
0011	0	0	4	6	0	0	3	2					
0111	0	6	5	6	0	2	1	2					
1111	7				1								

Hình 1: Minh họa Dynamic

1. Đầu tiên xét các tổ hợp 1 bao gồm B, C, D. Điền khoảng cách từ 3 tổ hợp 1 này tới điểm khởi đầu.
2. Tiếp theo xét tổ hợp 2 bao gồm BC, BD, CD. Trong đó tổ hợp 2 là kết quả từ 2 tổ hợp 1 phát triển lên như tổ hợp BC là BCA và CBA. Do 2 tổ hợp con không xung đột với nhau, các giá trị chưa cần phải so sánh ta tính giá trị và điền khoảng cách tính được vào ma trận X[0110] và ma trận Y[0110] theo 2 điểm nối B và C.
3. Thực hiện tương tự cho BD và CD
4. Phát triển tổ hợp 2 lên tổ hợp 3. Ta thấy từ 0110 lên 0111, D có thể chọn nối vào B hoặc C, ta chọn ra D nối với C trong tổ hợp BC cho khoảng cách nhỏ hơn ($3+3 < 2+5$). Gán giá trị vào X[0111][3] là 6 và Y[0111][3] là 2 (vì chọn C là điểm nối).
5. Thực hiện phát triển cho 2 tổ hợp 2 còn lại là BD và CD ta thu được các khoảng cách của tổ hợp 3 BCD.
6. Sau khi đã chọn đủ các thành phố, ta chọn ra đường đi ngắn nhất dựa trên khoảng cách từ A tới điểm nối + khoảng cách của tổ hợp. A nối đến B trong tổ hợp BCD cho ra khoảng cách ngắn nhất. Từ ma trận Y ta tìm lên các tổ hợp và thành phố đi qua, trong ví dụ này là Y[0111][1] (2) -> Y[0011][2] (3) -> Y[0001][3] -> Y[0000] kết thúc ra được kết quả A -> B -> C -> D.

2.3 Ant colony:

Đang thực hiện (khó giải thích và chưa tìm được ví dụ trực quan)



Thảo luận BTL môn KTLT, DSA, NMLT, PPL

<https://www.facebook.com/groups/211867931379013>

- Lớp BTL1 + GK + LAB + Lý thuyết + Harmony của môn DSA HK232
- **Lớp BTL2 + CK + LAB + Lý thuyết + Harmony của môn DSA HK232**
- Lớp BTL1 + Lý thuyết + Harmony của môn KTLT HK232
- **Lớp BTL2 + Lý thuyết + Harmony của môn KTLT HK232**
- **Lớp CK + LAB + Harmony của môn KTLT HK232**
- **BTL + CK + Harmony của môn CTRR HK232**
- Lớp BTL1 + BTL2 + GK + Harmony của môn PPL HK232
- **Lớp BTL3 + BTL4 + CK + Harmony của môn PPL HK232**

CHÚC CÁC EM HỌC TỐT

